

## Python Code

```

1  import csv
2  import io
3  import math
4  import sqlite3
5  import subprocess
6  import sys
7  import threading
8  from pathlib import Path
9  from flask import Flask, Response, jsonify, render_template, request, url_for
10 from linkedin_people_scraper import clean_profile_url, init_sqlite
11 BASE = Path(__file__).resolve().parent
12 DB_PATH = BASE / "all_people.db"
13 SCRAPER = BASE / "linkedin_people_scraper.py"
14 PER_PAGE = 25
15 CSV_FIELDS = ("first_name", "last_name", "job_title", "email")
16 REQUIRED_CSV_FIELDS = ("company_name", "linkedin_profile_url")
17 app = Flask(__name__)
18 # Single shared scrape job (only one browser session at a time).
19 job = {"proc": None, "running": False, "log": [], "returncode": None}
20 job_lock = threading.Lock()
21 # ----- #
22 # Database
23 # ----- #
24 def _escape_like(text: str) -> str:
25     """Escape LIKE wildcards so the search is a literal substring match."""
26     return text.replace("\\", "\\\").replace("%", "\\%").replace("_", "\\_")
27 def fetch_page(page: int, company_filter: str = "") -> tuple[list[dict], int, int]:
28     """
29     Return (rows, total_matching_rows, clamped_page) for the requested page,
30     optionally limited to rows whose company_name contains `company_filter`.
31     """
32     if not DB_PATH.exists():
33         return [], 0, 1
34     where = ""
35     params: list = []
36     if company_filter:
37         where = "WHERE company_name LIKE ? ESCAPE '\\'"
38         params.append(f"%{_escape_like(company_filter)}%")
39     con = sqlite3.connect(DB_PATH)
40     con.row_factory = sqlite3.Row
41     try:
42         total = con.execute(
43             f"SELECT COUNT(*) FROM employees {where}", params
44         ).fetchone()[0]
45     except sqlite3.OperationalError: # table not created yet
46         con.close()
47         return [], 0, 1
48     pages = max(1, math.ceil(total / PER_PAGE))
49     page = min(max(1, page), pages)
50     offset = (page - 1) * PER_PAGE
51     rows = con.execute(
52         f"""SELECT company_name, first_name, last_name, job_title,
53             linkedin_profile_url, email
54             FROM employees
55             {where}
56             ORDER BY company_name, last_name, first_name
57             LIMIT ? OFFSET ?"""
58         , ["params", PER_PAGE, offset],
59     ).fetchall()
60     con.close()
61     return [dict(r) for r in rows], total, page
62 # ----- #
63 # Scrape job management
64 # ----- #
65 def _reader(proc: subprocess.Popen) -> None:
66     """Stream the scraper's output into the job log."""
67     for line in proc.stdout:
68         with job_lock:
69             job["log"].append(line.rstrip())
70     proc.wait()
71     with job_lock:
72         job["running"] = False
73         job["returncode"] = proc.returncode
74 @app.route("/")
75 def index():
76     page = request.args.get("page", 1, type=int)
77     q = request.args.get("q", "").strip()
78     rows, total, page = fetch_page(page, q)
79     pages = max(1, math.ceil(total / PER_PAGE))
80     return render_template(
81         "index.html",
82         rows=rows,
83         total=total,
84         page=page,
85         pages=pages,
86         q=q,
87     )
88 @app.route("/scrape", methods=["POST"])
89 def scrape():
90     url = request.form.get("url", "").strip()
91     max_people = request.form.get("max_people", "100").strip()
92     if "linkedin.com/company/" not in url:
93         return jsonify(error="Please enter a LinkedIn company People page URL."), 400
94     if not max_people.isdigit() or int(max_people) < 1:

```

```

95     return jsonify(error="Max people must be a positive number."), 400
96 with job_lock:
97     if job["running"]:
98         return jsonify(error="A scrape is already running."), 400
99     cmd = [
100         sys.executable, "-u", str(SCRAPER), url,
101         "--sqlite", str(DB_PATH),
102         "--max-people", max_people,
103     ]
104     proc = subprocess.Popen(
105         cmd,
106         cwd=BASE,
107         stdin=subprocess.PIPE, # Lets the UI answer the login prompt
108         stdout=subprocess.PIPE,
109         stderr=subprocess.STDOUT,
110         text=True,
111         bufsize=1,
112     )
113     job.update(proc=proc, running=True, log=[], returncode=None)
114     threading.Thread(target=_reader, args=(proc,), daemon=True).start()
115     return jsonify(ok=True)
116 @app.route("/status")
117 def status():
118     with job_lock:
119         return jsonify(
120             running=job["running"],
121             returncode=job["returncode"],
122             log=job["log"][-200:],
123         )
124 @app.route("/continue", methods=["POST"])
125 def continue_scrape():
126     """Send Enter to the scraper after the user has logged in to LinkedIn."""
127     with job_lock:
128         proc = job["proc"]
129         if not job["running"] or proc is None:
130             return jsonify(error="No scrape is running."), 400
131         proc.stdin.write("\n")
132         proc.stdin.flush()
133     return jsonify(ok=True)
134 # ----- #
135 # CSV upload / download
136 # ----- #
137 @app.route("/upload_csv", methods=["POST"])
138 def upload_csv():
139     file = request.files.get("csv_file")
140     if not file or not file.filename:
141         return jsonify(error="Please choose a CSV file."), 400
142     try:
143         text = file.read().decode("utf-8-sig")
144     except UnicodeDecodeError:
145         return jsonify(error="Could not read the file as UTF-8 text."), 400
146     reader = csv.DictReader(io.StringIO(text))
147     if not reader.fieldnames:
148         return jsonify(error="The CSV file appears to be empty."), 400
149     header_map = {h.strip().lower() for h in reader.fieldnames if h}
150     missing = [f for f in REQUIRED_CSV_FIELDS if f not in header_map]
151     if missing:
152         return jsonify(
153             error=f"CSV is missing required column(s): {' '.join(missing)}"
154         ), 400
155     valid_rows = []
156     skipped = 0
157     for raw in reader:
158         values = {k.strip().lower(): (v or "").strip() for k, v in raw.items() if k}
159         company_name = values.get("company_name", "")
160         profile_url = clean_profile_url(values.get("linkedin_profile_url", ""))
161         if not company_name or not profile_url:
162             skipped += 1
163             continue
164         row = {
165             "company_name": company_name,
166             "linkedin_profile_url": profile_url,
167         }
168         for field in CSV_FIELDS:
169             row[field] = values.get(field) or None # blank/missing -> None
170         valid_rows.append(row)
171     if not valid_rows:
172         return jsonify(
173             error="No valid rows found (each row needs company_name and linkedin_profile_url).",
174         ), 400
175     init_sqlite(DB_PATH) # make sure the table exists
176     con = sqlite3.connect(DB_PATH)
177     existing_pairs = {
178         (c, u) for c, u in
179         con.execute("SELECT company_name, linkedin_profile_url FROM employees")
180     }
181     inserted = updated = 0
182     for row in valid_rows:
183         key = (row["company_name"], row["linkedin_profile_url"])
184         if key in existing_pairs:
185             updated += 1
186         else:
187             inserted += 1
188             existing_pairs.add(key)
189     con.executemany(
190         """INSERT INTO employees
191         (company_name, first_name, last_name, job_title,
192         linkedin_profile_url, email)

```

```

193         VALUES (:company_name, :first_name, :last_name, :job_title,
194                 :linkedin_profile_url, :email)
195     ON CONFLICT(company_name, linkedin_profile_url) DO UPDATE SET
196         first_name = COALESCE(excluded.first_name, employees.first_name),
197         last_name = COALESCE(excluded.last_name, employees.last_name),
198         job_title = COALESCE(excluded.job_title, employees.job_title),
199         email = COALESCE(excluded.email, employees.email)""",
200     valid_rows,
201 )
202 con.commit()
203 con.close()
204 return jsonify(ok=True, inserted=inserted, updated=updated, skipped=skipped)
205 @app.route("/download_csv")
206 def download_csv():
207     q = request.args.get("q", "").strip()
208     where = ""
209     params: list = []
210     if q:
211         where = "WHERE company_name LIKE ? ESCAPE '\\'"
212         params.append(f"%{_escape_like(q)}%")
213     output = io.StringIO()
214     writer = csv.writer(output)
215     writer.writerow(["company_name", "first_name", "last_name", "job_title",
216                    "linkedin_profile_url", "email"])
217     if DB_PATH.exists():
218         con = sqlite3.connect(DB_PATH)
219         try:
220             rows = con.execute(
221                 f"""SELECT company_name, first_name, last_name, job_title,
222                        linkedin_profile_url, email
223                     FROM employees {where}
224                     ORDER BY company_name, last_name, first_name""",
225                 params,
226             ).fetchall()
227             writer.writerows(rows)
228         except sqlite3.OperationalError:
229             pass
230         con.close()
231     filename = "employees_filtered.csv" if q else "employees.csv"
232     return Response(
233         output.getvalue(),
234         mimetype="text/csv",
235         headers={"Content-Disposition": f"attachment; filename={filename}"},
236     )
237 if __name__ == "__main__":
238     app.run(debug=False, port=5000)

```